

Other pygwidgets Classes

As you saw at the beginning of this section, `pygwidgets` contains a number of other classes.

The `ImageCollection` class allows you to show any single image from a collection of images. For example, suppose you have images of a character facing front, left, back, and right. To represent all the potential images, you can build a dictionary like this:

```
imageDict = {'front':'images/front.png', 'left':'images/left.png',  
            'back':'images/back.png', 'right':'images/right.png'}
```

You can then create an `ImageCollection` object, specifying this dictionary and the key of the image you want to start with. To change to a different image, you call the `replace()` method and pass in a different key. Calling the `draw()` method at the bottom of the loop always shows the current image.

The `Dragger` class displays a single image but allows the user to drag the image anywhere in the window. You must call its `handleEvent()` method in the event loop. When the user finishes dragging, `handleEvent()` returns `True`, and you can call the `Dragger` object's `getMouseUpLoc()` method to get the location where the user released the mouse button.

The `Animation` and `SpriteSheetAnimation` classes handle building and showing an animation. Both require a set of images to iterate through. The `Animation` class gets the images from individual files, while the `SpriteSheetAnimation` class requires a single image with evenly spaced internal images. We'll explore these classes more fully in Chapter 14.

pygwidgets Example Program

[Figure 7-4](#) shows a screenshot of a sample program that demonstrates objects instantiated from many of the classes in `pygwidgets`, including `Image`, `DisplayText`, `InputText`, `TextButton`, `CustomButton`, `TextRadioButton`, `CustomRadioButton`, `TextCheckBox`, `CustomCheckBox`, `ImageCollection`, and `Dragger`.

The source of this example program can be found in the `pygwidgets_test` folder in my GitHub repository, <https://github.com/IrvKalb/pygwidgets/>.